
Coding Problem 1: The Logistics Bottleneck (CargoLogix System)

Scenario Background:

You are a backend database engineer for *CargoLogix*, a global shipping logistics platform. The platform relies on a PostgreSQL database with two massive tables:

1. couriers (columns: courier_id PK, company_name, contact_email)
2. shipments (columns: shipment_id PK, courier_id FK, status, dispatch_date)

The operations team uses a real-time Java dashboard to monitor failing deliveries. Every 5 seconds, the dashboard queries the database for a list of all shipments currently marked as 'DELAYED', along with the name of the courier responsible, sorted from the newest dispatch date to the oldest.

Because the shipments table has grown to 50 million rows, this query is currently causing a massive CPU bottleneck, taking 15 seconds to execute and crashing the dashboard.

Implementation Constraints & Requirements:

1. The Database Optimization (SQL & Indexing):

- **The Join:** Write the exact, optimized SQL SELECT query to fetch shipment_id, company_name, and dispatch_date for all 'DELAYED' shipments, sorted by dispatch_date descending.
- **The Index:** Write the specific Data Definition Language (DDL) command to create an index on the shipments table that entirely eliminates the need for a full table scan and an in-memory sort for this specific query.

2. The Java Application (JDBC & Inheritance):

You must write the backend Java code to execute this query, strictly adhering to the following Object-Oriented design:

- **Interface:** Create an interface ReportGenerator with a method void printDelayedReport().
- **Base Class:** Create an abstract class DatabaseRepository. It must encapsulate the database credentials and contain a protected Connection getConnection() throws SQLException method that uses DriverManager.
- **Derived Class:** Create a LogisticsRepository class that extends DatabaseRepository and implements ReportGenerator.
- **JDBC Mechanics:** Inside printDelayedReport(), you must use modern JDBC features. You must use a PreparedStatement (even if there is no user input, to pre-compile the query), process the ResultSet, and guarantee that all database resources (Connection, Statement, ResultSet) are safely closed without causing memory leaks, even if an exception occurs.

Why this makes a great interview/study question:

1. **The Composite Index Strategy:** Many developers would just create an index on status. However, because the query also requires sorting (ORDER BY dispatch_date DESC), the database would still have to perform an expensive in-memory sort on millions of 'DELAYED' rows. A composite index specifically ordered (status, dispatch_date DESC) allows the B-Tree to be physically structured exactly how the query requests it. The database just jumps to the 'DELAYED' section of the index and reads it top-to-bottom.
2. **Architectural Separation (OOP):** By forcing the DatabaseRepository base class, it tests the candidate's understanding of separating infrastructure logic (connecting to the DB) from business logic (executing the specific report). The child class simply inherits the ability to connect.
3. **JDBC Best Practices:** The prompt specifically tests knowledge of **Try-with-Resources** (Java 7+). Prior to this feature, developers had to write massive finally blocks checking if (rs != null) rs.close(); which was prone to memory leaks (Connection Leaks) if forgotten. The try(...) syntax handles the exact teardown sequence automatically.
4. **PreparedStatement over Statement:** Using PreparedStatement with ? bind variables ensures the database pre-compiles the execution plan, and mathematically neutralizes SQL injection risks, which is an absolute must-know for backend integration.

Coding Problem 2: The "Edixo" Enrollment Engine (Missing Records & Batching)

Scenario Background:

You are the lead database architect for Edixo, a fast-growing EdTech startup managing university curriculums. The platform relies on a PostgreSQL database with two primary tables:

- **students** (columns: `student_id` PK, `full_name`, `university_email`)
- **course_registrations** (columns: `registration_id` PK, `student_id` FK, `course_code`, `semester`)

At the start of the semester, the administration needs a report of "At-Risk" students—specifically, students who are enrolled in the university but have absolutely **zero** course registrations in the system. Following this report, the system must automatically assign a mandatory "Orientation 101" course to all of these at-risk students. Because the platform spans multiple universities, the student table has millions of rows.

Implementation Constraints & Requirements:

1. The Database Optimization (SQL & Indexing):

- **The Join:** Write the exact, optimized SQL SELECT query to fetch `student_id` and `full_name` for all students who do not have a corresponding record in the `course_registrations` table. You must use an outer join to achieve this.
- **The Index:** Write the DDL command to create an index that prevents the database from performing a full table scan on the massive `course_registrations` table when resolving the foreign key relationships for this query.

2. The Java Application (JDBC Batch Processing & Inheritance):

- **Interface:** Create an interface `RegistrationManager` with a method `void enrollAtRiskStudents()`.
- **Base Class:** Create an abstract class `DatabaseConnectionProvider` that encapsulates the connection infrastructure.
- **Derived Class:** Create an `EdixoRegistrationRepository` class that extends the base class and implements the interface.
- **JDBC Mechanics:** Inside `enrollAtRiskStudents()`, you must execute the SQL query to find the missing students. Then, you must use JDBC **Batch Processing** (`addBatch()` and `executeBatch()`) to efficiently insert the "Orientation 101" records for those students, drastically reducing network round-trips to the database. Guarantee all resources close safely.

Why this makes a great interview/study question:

- **The Anti-Join Pattern (LEFT JOIN ... IS NULL):** Finding missing records is a staple of enterprise reporting. Using NOT IN with subqueries can severely degrade performance if nulls are present in the dataset. The LEFT JOIN combined with IS NULL is the safest, most performant way to identify orphaned or missing relational data.
 - **The Un-Indexed Foreign Key Trap:** Most junior developers assume the database automatically indexes Foreign Keys. PostgreSQL does not. Asking for the specific DDL command proves the candidate understands that joins on un-indexed FKs will result in catastrophic sequential scans.
 - **JDBC Batch Processing:** Firing 50,000 individual INSERT statements over a network will bottleneck any application. Demonstrating addBatch() and executeBatch() separated by chunk thresholds (e.g., 1000 records) proves senior-level awareness of heap memory management and network latency.
-

Coding Problem 3: The FIRE Portfolio Aggregator (Transactions & Grouping)

Scenario Background:

You are developing the backend for a financial independence planning application. The database tracks users striving for FIRE (Financial Independence, Retire Early) and their diverse investment portfolios.

- investors (columns: investor_id PK, email, tax_residency)
- holdings (columns: holding_id PK, investor_id FK, asset_class, current_value)

A user wants to restructure their portfolio to optimize for taxes. Before the restructuring runs, the system must query the current total sum of their assets, grouped by asset_class (e.g., Equity, Debt, Real Estate). Then, it must automatically shift a predefined amount of value from 'Debt' to 'Equity'. Because this involves money, the read and the two subsequent updates must be absolutely atomic.

Implementation Constraints & Requirements:

1. The Database Optimization (SQL & Indexing):

- **The Join:** Write a SQL query using an INNER JOIN and GROUP BY that fetches the asset_class and the total current_value for a specific investor_id.
- **The Index:** Create a Composite Index that perfectly covers the WHERE clause and the GROUP BY operation, ensuring the database does not need to read the table data pages (an Index-Only Scan).

2. The Java Application (JDBC Transactions & Inheritance):

- **Interface:** Create an interface PortfolioManager with a method void restructurePortfolio(long investorId).
- **Base Class:** Create FinancialDatabaseConfig to manage the connection.
- **JDBC Mechanics:** Inside the implementation class, you must manually manage the database transaction. You must disable auto-commit (conn.setAutoCommit(false)), execute the SELECT aggregation, execute two UPDATE statements to shift the funds, and finally call conn.commit(). If *any* exception occurs, you must execute conn.rollback() in a catch block to prevent a partial loss of funds.

Why this makes a great interview/study question:

- **Index-Only Scans:** For data warehouses and financial apps, aggregations (SUM, AVG) are brutally slow. Testing the candidate's ability to create a "Covering Index" proves they

understand how to bypass the heap completely, answering the query directly from the B-Tree structure.

- **ACID Compliance in Java:** It forces the realization that JDBC runs in auto-commit mode by default. If the first update succeeds and the second fails, the user loses \$50,000. Managing `setAutoCommit(false)`, `commit()`, and `rollback()` is the absolute foundation of reliable enterprise Java development.

Coding Problem 4: The Himalayan Fleet Tracker (Window Functions & Mapping)

Scenario Background:

You are maintaining the backend for a high-altitude motorcycle tracking system used in the Himalayan regions (Leh-Ladakh).

- **riders** (columns: `rider_id` PK, `rider_name`, `bike_model`)
- **gps_pings** (columns: `ping_id` PK, `rider_id` FK, `latitude`, `longitude`, `recorded_at` TIMESTAMP)

Thousands of riders are pinging their GPS locations every 30 seconds. The emergency response dashboard requires a real-time list of the **absolute latest** GPS ping for every single active rider, joined with the rider's name and bike model.

Implementation Constraints & Requirements:

1. The Database Optimization (SQL & Indexing):

- **The Join:** Write the SQL query to fetch the latest ping per rider. Do NOT use a GROUP BY with MAX() because that makes it impossible to retrieve the matching latitude/longitude in the same row safely. You must use an INNER JOIN paired with a SQL Window Function (ROW_NUMBER() OVER (...)).
- **The Index:** Create the composite index that specifically optimizes filtering by rider and sorting the timestamps descending.

2. The Java Application (JDBC Object Mapping):

- **Interface:** Create an interface `TelemetryService` with `void printLatestLocations()`.
- **Base Class:** Create `FleetDatabaseConnection`.
- **JDBC Mechanics:** When processing the `ResultSet`, you must map the relational TIMESTAMP column strictly into the modern Java 8 `java.time.LocalDateTime` API (do not use the legacy `java.util.Date`), proving familiarity with modern Java-to-SQL type conversions.

Why this makes a great interview/study question:

- **The "Greatest-N-Per-Group" Problem:** A classic SQL trap. Using GROUP BY rider_id, MAX(recorded_at) forces developers to write a highly inefficient self-join back to the gps_pings table to get the coordinates. Using ROW_NUMBER() OVER(PARTITION BY...) is the elegant, industry-standard solution for analytic time-series queries.
- **Modern Date/Time Mapping:** Legacy java.sql.Timestamp and java.util.Date are fundamentally flawed, mutable, and not thread-safe. Forcing the use of rs.getObject(column, LocalDateTime.class) ensures the developer is utilizing JDBC 4.2+ standards.

Coding Problem 5: The Enterprise Job Queue (Concurrency & Partial Indexes)

Scenario Background:

You are developing an asynchronous background job processing system for a massive enterprise architecture.

- departments (columns: dept_id PK, dept_name)
- background_jobs (columns: job_id PK, dept_id FK, status, created_at)

There are 100 million records in the background_jobs table, but 99.9% of them have a status of 'COMPLETED'. The system relies on multi-threaded worker nodes constantly polling the database to find jobs where status = 'PENDING', joined with their department name.

If multiple threads run the same query, they might accidentally fetch and process the exact same job twice (a race condition).

Implementation Constraints & Requirements:

1. The Database Optimization (SQL & Indexing):

- **The Join & Concurrency:** Write a SELECT query with an INNER JOIN to fetch the oldest 'PENDING' job for the 'Engineering' department. To prevent race conditions, append the PostgreSQL-specific clause FOR UPDATE SKIP LOCKED.
- **The Index:** Create a **Partial Index**. Indexing 100 million rows when you only care about the 100 pending ones wastes disk space and slows down inserts. Write the DDL to index the table *only* where the status is 'PENDING'.

2. The Java Application (JDBC Processing):

- **Interface:** Create an interface QueueWorker with void processNextJob().
- **Base Class:** Create EnterpriseConnectionFactory.

- **JDBC Mechanics:** Execute the concurrency-safe SELECT query, retrieve the job_id, and then execute an UPDATE statement to change that specific job's status to 'PROCESSING'.

Why this makes a great interview/study question:

- **Database-Level Concurrency:** FOR UPDATE SKIP LOCKED is the secret weapon of high-performance database queues. Junior developers try to handle race conditions in Java memory (which fails across distributed servers). This question forces the developer to utilize the database engine's native locking mechanics.
- **Partial Indexes:** A fundamental enterprise optimization. Indexing a boolean or low-cardinality status column (like 'COMPLETED'/'PENDING') normally creates a massive, useless B-Tree. The WHERE clause in the CREATE INDEX statement creates an ultra-lightweight index that ignores dead rows.

Problem 1: The Atomic Order Orchestrator (JDBC Transaction Management)

Enterprise Scenario: An e-commerce backend processes customer orders. When a user checks out, the system must deduct the purchased quantity from the Products table and create a record in the Orders table. If the product stock drops below zero during deduction, the entire operation must fail, and no order record should be saved.

Task: Write a JDBC program that processes an order for a product. You must disable AutoCommit, execute both the UPDATE (inventory) and INSERT (order) statements within a single transaction boundary, and manually trigger a rollback() if a custom InsufficientStockException is thrown, ensuring strict ACID Atomicity.

Code Breakdown & Database Analysis:

- `conn.setAutoCommit(false)`: By default, JDBC wraps every single `executeUpdate()` in its own isolated transaction. Disabling this tells the database to hold all changes in a temporary staging area (maintaining Isolation and Consistency).
- `conn.rollback()`: If the stock is insufficient, the exception shifts execution to the catch block. The rollback command destroys the temporary staging area. The UPDATE is completely undone, guaranteeing the Atomicity of the transaction (all or nothing).

Problem 2: The Malicious Wildcard Injection (PreparedStatement Security)

Enterprise Scenario: A banking dashboard allows administrators to search for users by email. A junior developer used PreparedStatement but improperly concatenated the wildcard (%) characters directly into the SQL string. A malicious insider attempts to dump the entire user table by submitting ' OR '1'='1 into the search field.

Task: Write a JDBC program that executes a LIKE query. Demonstrate how string concatenation inside the SQL query defeats the purpose of a PreparedStatement. Then, fix the code by securely binding the wildcards inside the `setString()` method.

Code Breakdown & Database Analysis:

- **The Trap:** If you write `LIKE '%' + input + '%'`, the Java compiler combines the string *before* sending it to the database driver. The Database parses the ' OR '1'='1 as executable SQL, resulting in a full table dump.
 - **The Fix:** By leaving strictly ? in the query, the Database pre-compiles the execution plan. When `pstmt.setString(1, "%" + maliciousInput + "%")` is called, the JDBC driver sanitizes the entire block, instructing the DB to look for a user whose email literally contains the characters O, R, ', 1, =. Since no such email exists, the attack safely returns 0 rows.
-

Problem 3: The B-Tree Bypass Defect (Query Optimization)

Enterprise Scenario: An auditing application fetches all financial records created in the year 2024. The table has 10 million rows and a B-Tree Non-Clustered Index on the `created_at` column. However, the database is executing a massive "Full Table Scan", ignoring the index and causing the Java thread to time out.

Task: The legacy JDBC query is written as `SELECT * FROM Records WHERE YEAR(created_at) = 2024`. Explain why the B-Tree index is being bypassed. Rewrite the query in Java to utilize range operators so the database engine can traverse the index logically ($O(\log N)$) instead of sequentially ($O(N)$).

Code Breakdown & Database Analysis:

- **SARGable Queries:** For a query to use an index, it must be "Search Argument Able". Functions (like `YEAR()`, `UPPER()`, or `SUBSTRING()`) applied directly to the indexed column blind the optimizer.
- **The Fix:** By rewriting the query using bounds (`>=` and `<`), the database engine can instantly locate "2024-01-01" in the B-Tree in $O(\log N)$ time, and traverse the leaf nodes sequentially until it hits "2025-01-01", avoiding the other 9.9 million irrelevant rows entirely.

Problem 4: Relational Object Mapping (Solving the N+1 Query Problem)

Enterprise Scenario: A dashboard needs to display all Departments and their corresponding Employees. The junior developer executed `SELECT * FROM Departments`, looped through the `ResultSet`, and executed a secondary query `SELECT * FROM Employees WHERE dept_id = ?` for every single department. This "N+1 Problem" crashed the database under heavy load.

Task: Write a single JDBC query utilizing an `INNER JOIN` to fetch all data simultaneously. In Java, parse the flattened relational `ResultSet` and construct a hierarchical Object Graph (a `Map<Integer, Department>` where each `Department` contains a `List<Employee>`).

Code Breakdown & Database Analysis:

- **INNER JOIN:** The database executes the join using its internal optimizer, fetching all matches in a single network round-trip. This drops the network overhead from $O(N)$ calls down to $O(1)$ call.
- **The Java Map:** Because SQL returns flattened, tabular data, a Department with 5 employees will result in 5 rows with duplicated Department data. The `HashMap` checks `departmentGraph.get(deptId)`. If it exists, it skips creating a duplicate `Department` object and simply appends the `Employee` object to the existing collection, perfectly reconstructing the hierarchical relationship in memory.

Problem 5: Cross-Hierarchical Fraud Detection (Advanced SELF & LEFT Joins)

Enterprise Scenario: A forensic audit team is investigating payroll fraud. They suspect that certain mid-level managers are creating "Ghost Employees" who have no assigned department, yet are collecting paychecks under the manager's approval code.

Task: Write an advanced JDBC multi-table query. You must execute a SELF JOIN on the Employees table to link workers to their managers, followed by a LEFT JOIN on the Departments table. Return the names of the "Ghost Employees", their Manager's name, and their NULL department status.

Code Breakdown & Database Analysis:

- INNER JOIN Employees m ON e.manager_id = m.emp_id: The SELF JOIN treats the Employees table as two distinct logical entities (e for subordinate, m for manager). It successfully retrieves the manager's actual name rather than just their numeric ID.
- LEFT JOIN Departments d: We use a LEFT JOIN because an INNER JOIN would automatically filter out any employee lacking a dept_id. By using LEFT, the ghost employee is kept in the result set, and the database generates NULL for their missing department data.
- WHERE e.dept_id IS NULL: This acts as a strict filtering pass over the joined dataset, isolating the exact anomalous records the audit team requires.